Solution Architecture — Retail Store Pricing Management System

1. Purpose

This document describes the proposed technical architecture for a web application that manages retail store pricing feeds. The system supports CSV upload, pricing-record search, and editing and saving existing records. The target operating scale is approximately 3,000 stores across multiple countries.

2. Architecture Overview

The solution uses a three-tier architecture:

- **Presentation tier:** React Single Page Application (SPA), used by authorised users in a web browser.
- **Application tier:** FastAPI service, responsible for request validation, CSV processing, business rules, search, and updates.
- **Data tier:** PostgreSQL database, responsible for persistent storage of pricing records.

The browser communicates with FastAPI over HTTPS. FastAPI communicates with PostgreSQL over a private database connection. The browser does not connect directly to the database.

3. Main Components

3.1 React Frontend

Responsibilities: - Provide a CSV upload interface. - Display upload progress, validation results, and processing status. - Provide search and filtering controls for pricing records. - Display matching records with pagination. - Allow authorised users to edit a record and submit changes. - Show success and error messages returned by the API.

3.2 FastAPI Backend

Responsibilities: - Expose REST API endpoints for CSV upload, search, record retrieval, and updates. - Validate file type, required columns, field formats, and business rules. - Parse CSV files and report invalid rows clearly. - Apply server-side validation to all edits. - Use parameterised database operations or an ORM to access PostgreSQL safely. - Return consistent HTTP status codes and structured error responses. - Record operational errors and, if required, auditable pricing changes.

3.3 PostgreSQL Database

Responsibilities: - Persist pricing records with Store ID, SKU, Product Name, Price, and Date. - Enforce appropriate data types, required fields, and constraints. - Support indexed searches, including store, SKU, product, and date criteria as appropriate. - Use transactions for reliable inserts and updates. - Support backup and recovery procedures.

4. Request and Data Flows

4.1 CSV Upload

1. A user selects a pricing CSV file in the React application.
2. React sends the file to the FastAPI upload endpoint over HTTPS.
3. FastAPI checks the file and required columns, then validates each row.
4. Valid rows are written to PostgreSQL in a transaction or controlled batches.
5. FastAPI returns an upload summary, including accepted and rejected row counts and useful validation errors.
6. React displays the result to the user.

For large files, processing should use bounded batches rather than loading the entire file into memory. The final batch size and synchronous versus background processing approach should be chosen after testing representative file sizes.

4.2 Search Pricing Records

1. The user enters search criteria in React.
2. React sends the criteria and pagination parameters to FastAPI.
3. FastAPI validates the parameters and queries PostgreSQL.
4. PostgreSQL returns the matching page of records.
5. FastAPI returns the results and pagination metadata.
6. React displays the records.

Search requests should use pagination and suitable database indexes to avoid returning an unbounded result set.

4.3 Edit and Save a Pricing Record

1. The user edits a record in React and submits the changes.
2. FastAPI validates the record identifier and submitted fields.
3. FastAPI updates PostgreSQL within a transaction.
4. The API returns the updated record or a structured error.
5. React refreshes the displayed record and shows the outcome.

If multiple users may edit the same record, add optimistic concurrency control (for example, a version field or updated timestamp) to prevent silent overwrites.

5. Proposed Deployment View

- Serve the React build through a web server or static hosting service behind HTTPS.
- Run FastAPI as a separate application service behind a reverse proxy or load balancer.
- Keep PostgreSQL on a private network and restrict access to the backend service.
- Store credentials and secrets in environment-specific secret management, not in source control.
- Use separate development, test, and production environments.
- Configure centralised application logs, health checks, database backups, and monitoring.
- Scale the stateless FastAPI service horizontally if measured demand requires it.

The specific hosting provider, number of instances, sizing, and availability topology are deployment decisions to be confirmed against budget, operational support, and service-level targets.

6. Data and API Design Considerations

Pricing Record

The core record contains: - `store_id` - `sku` - `product_name` - `price` - `date`

Recommended implementation considerations: - Define the price column using a fixed-precision decimal type, not a floating-point type. - Store dates using a consistent database date type and define the meaning of the feed's Date field. - Confirm whether price currency must be stored, since stores span multiple countries. - Define the uniqueness rule for a pricing record, such as whether a store can have multiple prices for the same SKU on different dates. - Add created/updated timestamps and user attribution if auditability is required.

Example API Resources

- `POST /api/pricing-feeds` — upload a CSV file.
- `GET /api/pricing-records` — search and paginate pricing records.
- `GET /api/pricing-records/{record_id}` — retrieve a single record.
- `PUT /api/pricing-records/{record_id}` — validate and save an edited record.
- `GET /health` — application health endpoint.

These are proposed endpoint names; align them with the existing codebase before implementation.

7. Security and Reliability

- Require authentication and role-based authorisation for upload and edit actions.
- Enforce HTTPS in deployed environments.
- Validate and limit uploaded file size and reject unexpected file types.
- Validate all input on the backend; frontend validation is for usability only.
- Use parameterised SQL or a well-configured ORM.
- Apply least-privilege database credentials and restrict network access.
- Avoid logging secrets or unnecessary sensitive data.
- Use transactions and clear error handling for data changes.
- Define backup retention and test restoration.
- Monitor API latency, errors, upload failures, and database resource usage.

Authentication, role management, audit history, monitoring, and deployment controls must be confirmed as implemented or treated as planned work.

8. Scaling for Approximately 3,000 Stores

The store count alone does not determine capacity. Workload depends on the number of records per feed, upload frequency, concurrent users, and search patterns. The architecture supports growth through: - Stateless API instances that can be scaled horizontally. - Paginated searches and targeted database indexes. - Batch CSV processing. - Connection pooling and query optimisation. - Load and stress testing using representative data volumes. - Database monitoring and capacity planning.

Partitioning, caching, background job queues, or read replicas should be introduced only if measurement shows they are needed.

9. Assumptions and Open Questions

- Users upload CSV files through the web application; automated store-to-system integration is not assumed.
- The required CSV columns are Store ID, SKU, Product Name, Price, and Date.
- The system must support stores in multiple countries.
- Currency handling, time zone rules, duplicate handling, and record uniqueness need confirmation.
- Authentication provider and user roles need confirmation.
- Maximum CSV size, upload frequency, concurrency, performance targets, availability targets, and recovery objectives need confirmation.
- Whether failed rows are rejected individually or cause the whole file to fail must be agreed.
- The proposed deployment environment and hosting provider have not yet been specified.

10. Architecture Summary

React provides the user interface, FastAPI validates requests and applies application rules, and PostgreSQL persists pricing data. The three-tier separation keeps database access behind the API and provides a practical base for secure operation and scaling. Final sizing and optional infrastructure components should be selected after workload requirements and performance tests are available.